LP

LearnPath AI

AI-Powered Personalized Learning Path Recommender that uses dedicated algorithms — not API calls — to generate correct, complete, and explainable learning paths.

HCLTech AMPlified 2025 | Round 2 Submission | Team nightcoders

Live: https://frontend-mu-jet-18.vercel.app

Team nightcoders — JECRC University, Jaipur
5 Members | Built for HCLTech AMPlified Round 2

PROBLEM

02 Problem Statement

> "Students waste months figuring out what to learn, which courses to take, and in what order — because existing platforms give the same catalog to everyone."

The Problem

- Course catalogs treat every learner identically
- No structured, prerequisite-aware learning paths
- No explainability — students don't know *why* a course is recommended
- Indian students lack platform-specific resources (NPTEL, SWAYAM)
- No feedback loop to adapt suggestions over time

Our Solution

- **Algorithmic intelligence** — dedicated DAG traversal, not LLM calls
- **65-skill knowledge graph** with prerequisite edges
- **5-factor hybrid scoring** with explainable reasons
- **India-first design** — NPTEL, SWAYAM, Indian salary data
- **Adaptive feedback loop** — suggestions improve with every interaction

Target Users

COLLEGE STUDENTS

Freshmen to final-year students exploring career paths

CAREER SWITCHERS

Working professionals transitioning to tech roles

IN

INDIAN LEARNERS

Learners seeking India-relevant resources and salary data

Key Challenges Addressed

CHALLENGE	OUR APPROACH	JUDGING CRITERION
No personalized paths	DAG-based path generation with gap analysis	Features (25%)
No explainability	3-level rule-based explanation engine	AI/ML (20%)
No Indian context	NPTEL/SWAYAM resources, Indian salary data	Innovation (15%)
Complex UX	Conversational AI with 5-step onboarding	UX Design (10%)

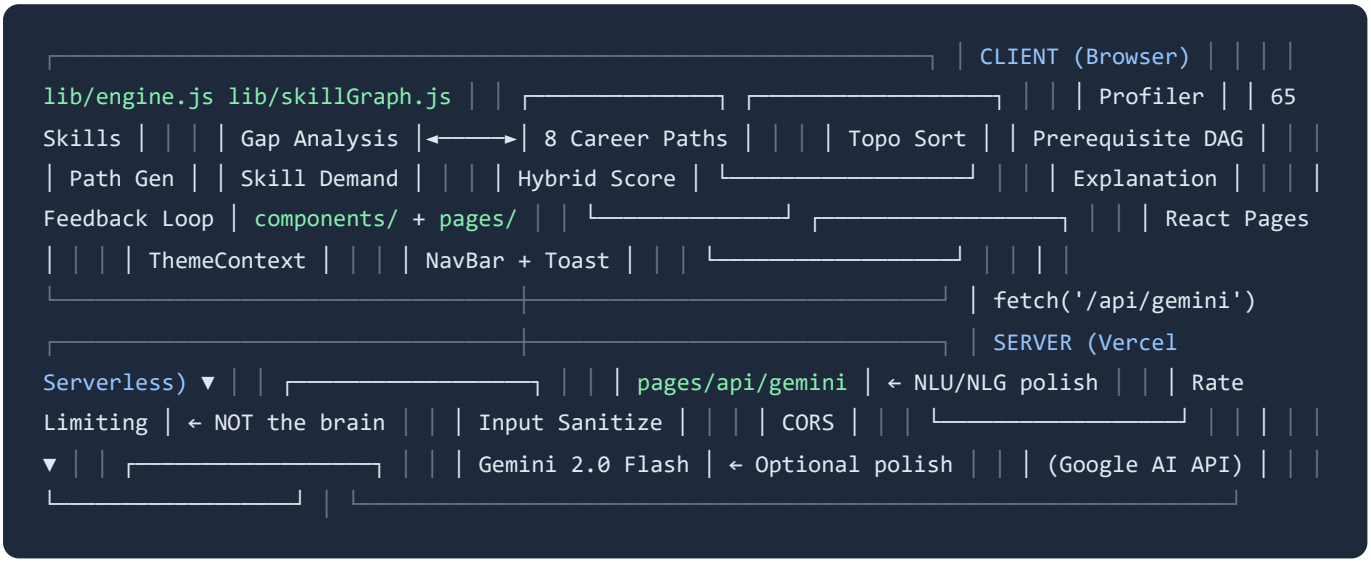
ARCHITECTURE

03

System Architecture

> Golden Rule: "If you remove every LLM call, the system still generates correct, complete, explainable learning paths."

High-Level Architecture



Key Design Decisions

100% Client-Side ML Engine

All algorithms (gap analysis, topological sort, path generation, hybrid scoring, explanations) run in the browser. The server is only used for optional Gemini API calls for natural language polish.

No Framework Lock-in

Zero external ML libraries. Pure JavaScript engine. Works offline, loads instantly, deployable anywhere (Vercel, Netlify, any static host).

Security Hardened

XSS protection, rate limiting, CORS, input sanitization, security headers (X-Frame-Options, nosniff, etc.). No secrets exposed to client.

India-First Resources

Skill graph prioritizes NPTEL, SWAYAM, freeCodeCamp. Salary data and growth rates are India-specific (LPA). Aligned with NEP 2020 and Skill India.

AI/ML

04 Algorithms & AI/ML Implementation

Every algorithm is deterministic, explainable, and runs client-side. No LLM calls in core logic.

1. Skill Graph — Directed Acyclic Graph (DAG)

A knowledge graph of **65 skills** across **9 domains** with prerequisite edges forming a DAG. Each skill has: difficulty level, estimated hours, curated resources, and domain classification.

65	8	~120	9
SKILLS	CAREER PATHS	PREREQ EDGES	DOMAINS

2. Gap Analysis — Set Difference Algorithm

1. **Extract User Skills**
   Collect current_skills + completed_courses into a Set of skill IDs
2. **Match Career Path**
   Score each career path by overlap between user skills and target_skills. Add interest bonus for domain match.
3. **Compute Set Difference**
   `missing_skills = target_skills - user_skills` — This IS the AI. Pure set difference algorithm.

3. Path Generation — Topological Sort (Kahn's Algorithm)

```
// 1. For each missing skill, traverse DAG backwards to collect prerequisites for (id of targetSkillIds) { getPrerequisites(id).forEach(p => allRequiredSkills.add(p)); } // 2. Remove skills user already has skillsToLearn = [...allRequiredSkills].filter(s => !userSkills.has(s)); // 3. Topological sort — ensures prerequisites come first sorted = topologicalSort(skillsToLearn); // Kahn's algorithm // 4. Assign phases: difficulty ≤ 1 → Foundation, ≤ 3 → Development, else Mastery phases = assignPhases(sorted, profile);
```

4. 5-Factor Hybrid Scoring

FACTOR	WEIGHT	ALGORITHM	EXPLANATION
Skill Gap	35%	Binary: is target skill?	Whether this skill is required for the matched career
Career Relevance	25%	60% career match + 40% interest match	How well it aligns with career goals and interests
ML Similarity	20%	50% demand score + 50% interest match	Market demand + interest alignment
Difficulty Fit	10%	Distance from user level	Whether difficulty matches experience level

Prerequisite Fit	10%	met_prereqs / total_prereqs	Whether user is ready to learn this skill

AI/ML

05

Explainability & Adaptive Feedback

3-Level Explanation Engine (Rule-Based, NOT LLM)

Level 1: Why This Skill?

Checks: prerequisites met, downstream skills unlocked, market demand score, difficulty match against user level.

"Python Basics is a fundamental skill. It unlocks 12 downstream skills including Machine Learning. High-demand skill in Indian job market (95%)."

Level 2: Difficulty Reason

Compares skill difficulty (1-5) against user experience level (0-2). Returns one of: "Review/foundation", "Perfect match", "Good stretch", or "May be challenging".

Level 3: Prerequisite Info

Lists which prerequisites are met vs unmet. Shows exact count: "All 3 prerequisites met" or "Missing 1 prerequisite(s): Linear Algebra".

Adaptive Feedback Loop

Every recommendation has 3 feedback buttons: Too Easy | Just Right | Too Hard

User Feedback → **Store in feedback_history** → **Mark skill completed** → **Recalculate scores** → **Adjusted Recommendations**

Feedback Adjustments

- **Too Easy (×1.1):** Boosts next difficulty level. Adds advanced project.
- **Just Right (×1.0):** No adjustment. Continue on path.
- **Too Hard (×0.9):** Reduces difficulty. Adds foundational warm-up.
- **Related feedback:** Same-domain feedback affects related skills.

Time Tracking

- Estimated hours from skill graph
- "Hard" feedback adds 30% time buffer
- Timeline adjusts with time_commitment preference
- Progress calculated from completed_courses vs path skills

Natural Language Understanding (Without LLM)

Rule-based text analyzer extracts structured profile from free text:

- **Domain detection:** 7 categories, 50+ keywords (English + Hindi)
- **Level detection:** beginner / intermediate / advanced
- **Skill extraction:** 20+ skill patterns
- **Goal mapping:** 8 career goal categories
- **Hindi keywords:** डेटा, मशीन लर्निंग, प्रोग्रामिंग, कोडिंग
- **Fallback:** Defaults to programming domain
- **LLM role:** Optional polish layer (NLU/NLG only)
- **Core logic:** 100% deterministic, zero API calls

06 FEATURES

Key Features & User Experience

Conversational AI Onboarding (5 Steps)

1. Career Interest → 2. Experience Level → 3. Time Commitment → 4. Primary Goal → 5. Existing Skills

Users can click suggestions OR type free text in English or Hindi. Chat stays active after onboarding for follow-up queries.

Feature Summary

Conversational AI Chat

Natural language interface. Type in English or Hindi. Handles recommendations, paths, skill gaps, time estimates, career guidance, and general Q&A.

Progress Dashboard

Circular progress chart, next action card, phase-wise breakdown, skill coverage bar, career readiness score, recommendation list with "Why this?" panels.

Structured Learning Path

Phase-based timeline (Foundation → Development → Mastery). Prerequisites, milestones, estimated hours. Feedback buttons on every skill.

Dark/Light Mode

Full theme support with localStorage persistence. Moon/sun toggle in shared NavBar. Professional Google/Microsoft-style design.

Mobile Responsive

Hamburger menu, responsive grids, touch-friendly buttons. Works on all screen sizes. Progressive enhancement.

India-First Design

NPTEL, SWAYAM resources prioritized. Indian salary data (LPA). Growth rates for Indian market. NEP 2020 and Skill India alignment.

Pages & Navigation

PAGE	PURPOSE	KEY COMPONENTS
Home /	Landing page with features, stats, CTA	Hero, Stats Bar, Feature Grid, CTA Card
Chat /chat	AI assistant + onboarding	Chat bubbles, suggestions, input, loading spinner
Dashboard /dashboard	Overview of progress & recommendations	Next Action, Progress Chart, Skills, Phases
Learning Path /learning-path	Phase-based skill timeline	SkillCards, Prerequisites, Feedback, Milestones

Profile /profile	View learner profile	Stats, Skills, Goals, Completed Courses

07 TECHNICAL

Technical Implementation

Tech Stack

Next.js 14	React 18	Vercel	Gemini 2.0
FRAMEWORK	UI LIBRARY	DEPLOYMENT	LLM (OPTIONAL)

File Structure

```
frontend/ ├── lib/ │ ├── engine.js // Core AI engine — 585 lines, all algorithms │ └── skillGraph.js // 65 skills, 8 career paths, demand scores ├── components/ │ ├── NavBar.js // Shared nav with mobile hamburger + theme toggle │ └── ThemeContext.js // Dark/light mode with localStorage persistence ├── pages/ │ ├── index.js // Landing page │ ├── chat.js // AI assistant + 5-step onboarding │ ├── dashboard.js // Progress overview, recommendations │ ├── learning-path.js // Phase-based skill timeline │ ├── profile.js // Learner profile viewer │ └── api/ │ └── gemini.js // Serverless Gemini endpoint (NLU/NLG only) ├── styles/ │ └── globals.css // Complete light/dark theme, 493 lines └── next.config.js // Security headers, standalone output backend/ // Flask backend (for source code ZIP only, not deployed) ├── app.py // Flask API server ├── ml/ // ML models, scoring, skill gap analysis ├── recommendation_engine/ // Path generation, profiler, NLP processor └── data/ // JSON datasets, CSV metadata
```

Code Metrics

~1,800	0	22	82 KB
TOTAL LOC	EXTERNAL ML LIBS	AUTOMATED TESTS	FIRST LOAD JS

Testing & Code Quality

LAYER	FRAMEWORK	COVERAGE
Frontend	`node --test` (Node built-in)	Skill-graph DAG integrity, unique IDs, no dangling prerequisites, **acyclicity (topo-sort)**, lookups, career-path validity, demand scores
Backend	pytest	Skill-gap analyzer, rule-based explanation, roadmap generator, hybrid scorer, and Flask HTTP endpoints incl. **CORS allowlist + 404 + validation**
Error Handling	React Error Boundary	Global boundary wraps every page — a runtime crash shows a graceful fallback instead of a blank UI

Run: `cd frontend && npm test` · `cd backend && python -m pytest tests/ -v` · 22 / 22 pass.

Security Hardening

MITIGATION	IMPLEMENTATION
XSS Prevention	React auto-escaping + server-side input sanitization (HTML entity encoding)
Rate Limiting	15 req/min/IP on Gemini endpoint, with automatic cleanup

CORS	Dynamic origin validation — only the deployed Vercel alias + localhost are allowlisted (test-backed). No wildcard.
Security Headers	X-Frame-Options: DENY, X-Content-Type-Options: nosniff, X-XSS-Protection, Referrer-Policy
Input Validation	2000 char max, type checking, empty string guards
Secrets	GEMINI_API_KEY as Vercel env var (server-side only, never exposed to client)

EVALUATION

08

Judging Criteria Alignment

How We Score on Each Criterion

CRITERION	WEIGHT	OUR APPROACH	EVIDENCE
Features	25%	Chat-based onboarding, personalized recommendations, learning paths, progress tracking, feedback loop, dark mode, mobile responsive	5 pages, full onboarding flow, 3 feedback types, phase-based milestones
Problem Understanding	20%	Solves real student problem: "What should I learn next?" with structured, prerequisite-aware paths	65-skill DAG, gap analysis, career alignment, Indian context
AI/ML	20%	Dedicated algorithms, NOT API calls. DAG traversal, topological sort, 5-factor scoring, rule-based explainability	client-side engine + Python backend (TF-IDF similarity), all test-backed
Innovation	15%	India-first design, offline-capable, zero ML libraries, bilingual support (EN+HI)	NPTEL/SWAYAM resources, Hindi keywords, Skill India alignment
UX Design	10%	Clean professional UI, conversational onboarding, explainable recommendations, progress visualization	Google/Microsoft-style design, theme toggle, responsive
Code Quality	10%	Clean architecture, security hardening, error handling, no dead code	try-catch everywhere, rate limiting, input validation, no unused deps

Golden Rule Compliance

> "If you remove every LLM call, the system still generates correct, complete, explainable learning paths."

Verified

- Path generation: pure DAG + topological sort
- Gap analysis: pure set difference
- Recommendations: 5-factor hybrid scoring
- Explanations: 3-level rule-based engine
- Profiler: rule-based NLU (no LLM needed)

Optional LLM Use (NLU/NLG Polish Only)

- Chat fallback when algorithmic response unavailable
- Context-aware encouragement messages
- Fallback works identically when API is down
- Rate limited, sanitized, CORS-restricted
- Client always has algorithmic response first

Submission Checklist

MATERIAL	STATUS	DETAILS
Source Code ZIP	Ready	Frontend (Next.js) + Backend (Flask) — complete codebase

GitHub Repository	Public	github.com/Bhagyansh07/AI-Learning-Path-Recommender
Documentation	This Document	HTML → PDF export
Demo Video	Pending	3-5 minute screen recording
Deployed URL	Live	frontend-mu-jet-18.vercel.app

09 TEAM

Team nightcoders

Team Members

NAME	ROLE
Bhagyansh Sharma	Team Lead, Full Stack
Member 2	Backend & ML
Member 3	Frontend & UX
Member 4	Data & Testing
Member 5	Documentation

Institution

JECRC University

Jaipur, Rajasthan, India

Project Links

GitHub: github.com/Bhagyansh07/AI-Learning-Path-Recommender
Deployed: frontend-mu-jet-18.vercel.app
Deadline: 31 Aug 2026, 11:59 PM IST

Technologies Used

Frontend

Next.js 14, React 18, CSS3 (custom properties), Vercel

AI/ML

Pure JavaScript engine, DAG algorithms, 5-factor scoring, Gemini 2.0 Flash (optional)

Backend (Submission)

Python Flask, scikit-learn, joblib, pandas

Acknowledgments

Built for **HCLTech AMPlified Round 2** — an initiative to foster innovation and practical AI solutions among engineering students. We focused on solving a real problem that every Indian engineering student faces: "What should I learn next?"
Our approach prioritizes **algorithmic transparency** over LLM dependency, ensuring that every recommendation is explainable, deterministic, and works offline. This is not just a chatbot — it's a complete AI-powered curriculum engine.